HacdsDeFi RPC 文档

2026-10-21　面向开发者

0. 服务一览

hacds 涉及三个 HTTP 服务，各自独立，可以分开部署。

服务	默认地址	协议	作用
全节点	`43.251.101.101:8082`	HTTP/JSON	查链上数据、沙箱只读调用、校验/广播交易
预言机	`43.251.101.101:8123`	HTTP/JSON	签发钻石评分证书
索引器	`43.251.101.101:8124`	HTTP/JSON	借款人台账（谁可清算）

钱包不本地扫链，所有链上数据都通过这三个服务拿。

```
全节点 :8082 ──GET──► 索引器 :8124 ──GET /watchlist──► 客户端
     ▲                                                  │
     └──────────────── 预言机 :8123（签证书）◄──────────┘
```

响应约定：所有接口返回 JSON，顶层带 `ret` 字段。`ret: 0` 表示成功，非 0 表示失败（失败时通常带 `err`）。

注意：失败也是 **HTTP 200**，必须判 body 里的 `ret`，不能只看状态码。

---

1. 全节点 API

1.1 查询区块高度

```
GET /query/block/height/latest
```

```
{"ret":0,"height":792580}
```

1.2 查询平均费率

用于估算手续费。

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size（字节）`，节点准入要求 `纯度 ≥ min_purity`（当前 `6024000000`）。

1.3 查询余额（HAC / 钻石 / 资产）

```
GET /query/balance?address={地址}&assets=true&asset_meta=true&diamonds=true
```

三个开关都是可选的：

参数	作用
`assets=true`	返回 `assets` 数组（链上资产）
`asset_meta=true`	`assets` 里附带 `name` / `ticket` / `supply` / `decimal`
`diamonds=true`	返回 `diamonds`（钻石名拼接串，每 6 字符一颗）

实测响应（地址 `15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`）：

```
{
  "ret": 0,
  "list": [{
    "hacash": "217001914996:237",
    "diamond": 7,
    "satoshi": 0,
    "diamonds": "ZUKKXZTTBAAUAYEWKKIXAYYUYTXNNBEUETIVYBYHYX",
    "assets": [{
      "serial": "2000",
      "amount": "4000000000",
      "decimal": 8,
```

```
      "name": "hacds",
      "ticket": "HACDS",
      "supply": "5963298700000000"
    }]
  }]
}
```

字段说明：

字段	含义
`hacash`	HAC 余额，格式 `值:单位`（`237` 表示 1:237 精度）
`diamond`	钻石数量
`diamonds`	钻石名拼接串，**每 6 个字符一颗**
`satoshi`	SAT 余额
`assets[].serial`	资产序列号。**hacds** = `2000`
`assets[].amount`	余额。字符串，避免 JS 精度丢失
`assets[].decimal`	小数位。hacds 是 `8`
`assets[].supply`	总供给

⚠ **amount 是 atoms，不是个数。** `4000000000` = 40 个 hacds。见第 4 节。

1.4 查询钻石详情

```
GET /query/diamond?name={6位钻石名}
```

```
{
  "ret": 0,
  "name": "KWTVSH",
  "belong": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "inscriptions": ["hacds"],
  "number": 85662,
  "miner": "1GSL5G6LyuKgg325DxPtfPouaTeTHaZbGh",
  "born": {"height": 522125, "hash": "00000000..."},
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "visual_gene": "aab018d628cb912cc290"
}
```

`belong` 是当前持有者。若等于合约地址，说明这颗钻石已被存入合约。

`visual_gene` 是图样和评分的输入。

1.5 沙箱只读调用（读合约状态的主要方式）

```
GET /query/contract/sandboxcall?contract={合约}&function={函数}&params={参数}&caller={调用者}
```

这是只读执行，不写链上状态、不发交易、不花手续费。但对于**需要读取合约数据的调用，** `caller` **建议传合约地址本身**——否则部分函数会因调用者余额不足而失败（见下方注意）。

实测响应：

```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```

字段	含义
`ret`	0 = 调用成功
`ret_val`	函数返回值（u128，字符串或数字）
`use_gas` / `gas_use`	本次消耗，用于估算

⚠ **注意 1：不传** `caller` **会失败。** 实测不传时返回：

```
{"ret":1,"err":"[revert] address \"1111111111111111111114oLvT2\" balance 0:0 is insufficient, at least 559555:242"}
```

默认调用者的余额为 0 付不起预扣 gas。**传** `caller={合约地址}` **即可正常。**

⚠ **注意 2：每 IP 并发上限为 2。** 节点对沙箱调用有并发限制，超出的请求**直接拒绝**（不排队）：

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

客户端必须**串行化 + 失败重试**，不要 `Promise.all` 并发十几个。

合约只读函数清单

参数格式为 `值:类型`，多个用逗号分隔。

函数	参数	返回	说明
`amm_reserves`	—	u128	池中 HAC（**zhu**）
`amm_reserve_ds`	—	u128	池中 hacds（**atoms**）
`pool_price`	—	u128	池价 = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	LP 总份额
`lp_share_of`	`{addr}:address`	u128	某地址的 LP 份额
`lp_pending_hac`	`{addr}:address`	u128	待领 LP 奖励（HAC，zhu）
`lp_pending_ds`	`{addr}:address`	u128	待领 LP 奖励（hacds，atoms）
`total_staked`	`{kind}:u128`	u128	质押总量。kind `0` =HAC，`1` =hacds
`treasury_pending`	`{kind}:u128`	u128	金库待结算。kind `0` =HAC，`1` =hacds
`balance_hac`	`{addr}:address`	u128	合约内 HAC 积分（zhu）
`balance_ds`	`{addr}:address`	u128	待领取 hacds 配额 due（atoms）
`loan_status`	`{addr}:address`	u128	借款人状态，见下
`loan_debt_of`	`{addr}:address`	u128	债务（zhu）
`loan_coll_of`	`{addr}:address`	u128	抵押（atoms）
`quote_score`	`{name}:bytes,{vg}:bytes,{num}:u128`	u128	按名字+基因+编号查评分

`loan_status` **返回值语义**（重要，不要按「健康度百分比」理解）：

返回值	含义
`0`	无借贷、无抵押
`999`	有抵押但无债务（安全）
其它	`coll_hac × 100 / debt`，即**抵押率 ×100**

清算判据：`debt > 0 && 0 < status < 200` 时可清算。

实测样本：

```
GET /query/contract/sandboxcall?contract=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
    &function=amm_reserves&caller=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
→ {"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}

GET ...&function=loan_status&params=15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q%3Aaddress&caller={合约}
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":0}
```

> `:` 必须 URL 编码为 `%3A`。

1.6 查询区块概要

索引器用它遍历区块找借款人。

```
GET /query/block/intro?height={高度}&tx_hash_list=true
```

```
{
  "ret": 0,
  "hash": "0000000000279d55a83105ce01311cd50b06fb5efcb1175e3891a9b46fcf27b1",
  "version": 1,
  "height": 792575,
  "timestamp": 1792537829,
  "nonce": 17,
  "difficulty": "...",
  "miner": "...",
  "reward": "...",
  "tx_hash_list": ["b6488be2b07be8745ac5827e00928ebc840a803d6a132cd9eb573812221e55ce"]
}
```

`tx_hash_list` 只在带 `tx_hash_list=true` 时返回。

1.7 查询交易

```
GET /query/transaction?hash={交易哈希}&action=true
```

不带 `action=true` 时只返回概要：

```
{
  "ret": 0,
  "hash": "b6488be2...",
  "hash_with_fee": "cf947b86...",
  "type": 2,
  "timestamp": 1792537576,
  "fee": "245:247",
  "fee_got": "245:246",
  "main_address": "1NsStAq1xt7cuqvEAobcS8T6gqcyS2gnMB",
  "action": 1,
  "block": {"height": 792575, "timestamp": 1792537829},
  "confirm": 5
}
```

带 `action=true` 时多出结构化动作数组：

```
"actions": [{
  "kind": 4,
  "d": {
    "diamond": "IKEMYN",
    "number": 138592,
    "prev_hash": "0x0000...",
    "nonce": "0x00000001a90d5c45",
    "address": "1EaqdAi9FLCQwLYPLWVtoRM1fZwGezXLTi",
    "custom_message": "0xb0114b5f..."
  }
}]
```

合约调用是 `kind = 44`，其字节码里含函数 selector：

```
09 14 01 <sig4: 4字节> ee
```

`sig4 = sha3_256(函数名)[0:4]`。索引器靠它识别「这笔交易调用了哪个合约函数」。

注意：交易确认后（`confirm ≥ 1`）此接口才能查到，未上链的查不到。

1.8 校验交易（附带人工可读描述）

```
POST /util/transaction/check?description=true
Content-Type: application/octet-stream
Body: 交易的二进制字节（不是 hex 字符串）
```

签名与否都能校验。 带 `description=true` 会返回人话描述。

实测响应：

```
{
  "ret": 0,
  "hash": "37637b89...",
  "hash_with_fee": "14f0c5e3...",
  "type": 3,
  "timestamp": 1792469415,
  "fee": "1:248",
  "fee_got": "1:248",
  "main_address": "15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q",
  "action": 1,
  "gas_max": 99,
  "description": "Main account 15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q pay 1:248 HAC tx fee",
  "actions": [{
    "action": {
      "kind": 16,
      "metadata": {
        "serial": 2000,
        "supply": 5963298700000000,
        "decimal": 8,
        "issuer": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
        "ticket": "0x4841434453",
        "name": "0x6861636473"
      },
      "protocol_cost": "8:248"
    },
    "description": "Register Asset #2000 \"HACDS\" (name \"hacds\", supply 5963298700000000 atoms, 8 decimals, issuer VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1);
pay 8 HAC protocol fee"
```

```
  }]
}
```

失败时同样 HTTP 200：

```
{"ret":1,"err":"transaction body invalid"}
```

1.9 广播交易

```
POST /submit/transaction
Content-Type: application/octet-stream
Body: 已签名交易的二进制字节
```

成功：

```
{"ret": 0, "hash": "交易哈希"}
```

失败：

```
{"ret": 1, "err": "transaction parse failed"}
```

⚠ `ret:0` **只表示进入内存池，不代表已上链。** Hacash 出块目标 **300 秒**，要等下一个块才生效。

需要确认时轮询 `/query/transaction?hash=...` 看 `confirm` 字段，或用 `/query/balance` 看余额变化。

1.10 合约存储费参数

部署合约、或估算新建键成本时用。

```
GET /query/contract/storage_fee
```

```
{
  "ret": 0,
  "enabled": true,
  "rule_version": 1,
  "head_height": 792581,
  "target_capacity_blocks": 1000,
  "capacity": 1000000,
  "rate": 1000,
  "remaining": 1000000,
  "p_min": 10,
  "p_max": 10000,
  "periods": 10,
  "k_max": 16384
}
```

`periods` 是当前折扣期数。若不为满价，生成部署动作时要按它重算成本。

---

2. 预言机 API

2.1 健康检查

```
GET /health
```

```
{"ok": true, "oracle": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu"}
```

`oracle` 是预言机账户地址。**这个地址在合约里是固定的**，换成别的地址签的证书合约不认。

2.2 申请评分证书

兑换 / 赎回钻石时必需。

```
GET /cert?name={6位钻石名}
```

实测响应：

```
{
  "name": "KWTVSH",
  "number": 85662,
  "visual_gene": "aab018d628cb912cc290",
```

```
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "score": 20,
  "hacds_amount": 2000000000,
  "details": {
    "shapes": "Common diamond",
    "styles_or_color": "Double mix",
    "nameliteral": "Common diamond",
    "numberliteral": "Common diamond",
    "score": 20
  },
  "cert_message_hex": "4b5754565348aab018d628cb912cc2900000000000014e9e0000001401",
  "cert_sha3_256": "0fca486f6d7c934006efb7ab926a12928bfda49d09ced3440c27ac2c39a88921",
  "sign_hex":
"023d7424448df611baf27f5188d1e48450082e94bb4d207b1612e22d299bd372d1aff139d57bd79446670c3ef77bab7471a33ada1df5f72adc90e13d316d7bcffc4ce07221ae7730e4b84b51f6320
  "oracle_address": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu",
  "version": 1
}
```


字段	含义
`score`	钻石评分。**1 分 = 1 个 hacds**
`hacds_amount`	能换的 hacds，单位 **atoms**（`2000000000` = 20 个）
`cert_message_hex`	证书消息体（29 字节），进交易的 Message/Blob
`sign_hex`	预言机对该消息的签名（97 字节）
`oracle_address`	签名账户

证书长度 = 126 字节（消息 29 + 签名 97）。一笔交易里放 N 颗钻石的证书，就是 `N × 126` 字节。

错误：

```
{"error": "query name must be 6 chars"}   // HTTP 400
{"error": "..."}                          // HTTP 502，多半是节点不可达
```

⚠ **钻石名必须 6 个字符**，不区分大小写（服务端会转大写）。

---

3. 索引器 API

索引器只读消费全节点，**从不提交交易、不需要私钥、不需要余额。**

它解决一个问题：**合约枚举不出借款人。**借款账目按 `loan:debt:<地址>` 拼键存储，合约没有列表、不写日志、节点也不支持按前缀枚举合约 KV。所以只能遍历区块，把历史调用过借贷函数的地址挖出来做成台账。

> **部署状态说明**：当前 `43.251.101.101:8124` 返回 `502`，服务未在运行。
> 下面的响应结构取自源码（`hacds_indexer.py`），启动服务后即为此形态。

3.1 健康检查

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 792580, "watched": 12}
```

3.2 台账快照（客户端唯一需要的接口）

```
GET /watchlist
```

```
{
  "ret": 0,
  "head": 792580,
  "node": "http://43.251.101.101:8082",
  "contract": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "watched": 12,
  "withDebt": 3,
  "liquidatable": 1,
  "snapshot": [{
    "address": "1Abc...",
    "health": 150,
    "healthPct": 1.5,
    "debt": 12345,
    "coll": 67890,
    "liquidatable": true,
    "noLoan": false,
    "first": 792000,
    "last": 792575,
```

```
    "calls": 4,
    "fns": {"loan_borrow": 2, "loan_add_collateral": 2},
    "manual": false,
    "note": null
  }]
}
```

字段	含义
`head`	已扫描到的高度
`watched`	台账地址数
`withDebt`	其中有债务的
`liquidatable`	其中可清算的
`snapshot[].health`	`loan_status` 原值（`0` / `999` / 抵押率×100）
`snapshot[].healthPct`	换算成倍数（`1.5` = 150%）。`0` 和 `999` 时为 `null`
`snapshot[].liquidatable`	服务端已按 `debt > 0 && 0 < health < 200` 判好
`snapshot[].first/last`	首次 / 最近一次出现在借贷函数调用里的块高
`snapshot[].fns`	调用过的函数计数
`snapshot[].manual`	是否手工添加

客户端不需要自己算可清算性，直接用 `liquidatable` 即可。

3.3 仅快照

```
GET /snapshot
```

```
{"ret": 0, "list": [ /* 同上 snapshot 数组 */ ]}
```

3.4 立即同步

```
GET /sync
```

触发一次扫块，返回本次结果。

3.5 手工维护台账

合约上线早期没有历史可扫，或需要补充观察地址时用。

```
GET /watch/add?address={地址}&note={备注}
GET /watch/remove?address={地址}
```

`/watch/add` 成功返回更新后的地址列表：

```
{"ret": 0, "list": ["1Abc...", "1Def..."]}
```

`/watch/remove`：

```
{"ret": 0, "removed": true}
```

缺少 `address` 时返回 HTTP 400：

```
{"ret": 1, "err": "address required"}
```

⚠ **台账要唯一。** 如果同时跑多个索引器实例，`/watch/add`、`/watch/remove` 也要走同一台，否则各处台账不一致。

3.6 部署

```
python3 hacds_indexer.py --port 8124 --node http://<node>:8082
```

- 零第三方依赖，Python 3 标准库
- 支持 `--workers` 并发扫块（默认 4，实测 2.6 → 6.6 块/秒）
- 增量同步：记 `lastScanned`，之后每轮只扫新增块
- 台账原子落盘（`tmp` + `os.replace`）
- systemd 单元见 `hacds_exchange/indexer/hacds_indexer.service`

---

4. 交易所上币对接（专章）

这一节是给要接入 hacds 的交易所 / 钱包 / 行情服务看的。

4.0 先记住一件事：amount 是 atoms

hacds 是 `decimal = 8` 的资产。链上和接口里的 `amount` 是**最小单位 atoms**，不是「个数」。

```
1 个 hacds = 100000000 atoms（1e8）
总供给 = 5963298700000000 atoms = 59,632,987 个 hacds
```

这一条搞错就是差一亿倍。

4.1 资产身份（用于上币登记）

项	值
serial	**2000**
ticket	`HACDS`
name	`hacds`
decimal	**8**
总供给	`5963298700000000` atoms
issuer	`VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1`
发行方地址	`15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`

⚠ 链上另有一个 ticket 也叫 `HACDS` 的资产（serial `1028`，name 是 "HACD Fragments"，与我们无关）。

协议不检查 ticket / name 唯一性，只检查 serial 是否被占用。

接入时必须以 `serial = 2000` 为准，不要按 ticket 匹配。

4.2 充值识别

不要按 serial 匹配，要按 serial=2000 + issuer 双重校验。

轮询或用节点推送拿区块，解析交易里的资产动作：

```
GET /query/block/intro?height={高度}&tx_hash_list=true
  → tx_hash_list
GET /query/transaction?hash={哈希}&action=true
  → actions[]，筛出资产转移
```

资产转移的 action `kind`：

kind	动作	含义
17	`TransferAssetTo`	转给合约/简单目标
19	`TransferAssetFromTo`	从 A 转给 B

从 action 的 `d` 字段里取 `serial` / `amount` / 收发地址。**`serial == 2000` 才是 hacds**，且建议同时校验 `issuer` 是合约地址。

入账金额：

```
用户看到的 hacds 个数 = amount / 1e8
```

例如收到 `amount = 4000000000`，入账 **40 个 hacds**。

⚠ 用大整数。 `5963298700000000` 已超过 JS `Number.MAX_SAFE_INTEGER`（`9007199254740991`）的 66%。JSON 里的 `amount` 是字符串，**不要先 parse 成 double。** JS 用 `BigInt`，Go 用 `big.Int`，Java 用 `BigInteger`，Kotlin 用 `BigInteger`（`Long` 不够）。

4.3 提现构造

hacds 是标准同态化资产，提现就是一次普通的资产转账（`kind = 19`，`TransferAssetFromTo`）。

参数：

参数	值
`serial`	`2000`
`amount`	要提的 atoms（= 个数 × 1e8）
`from`	交易所热钱包地址
`to`	用户地址

注意：`from` 必须是交易发起账户，或已获授权；交易所需要自己管好热钱包私钥并在本地签名。

构造方式有两种：

1. **用 Hacash 官方工具**自己拼 Type-3 交易
2. **参考本仓库的 Rust 引擎** `hacds_txbuild`（`transfer asset-from-to` 子模式），源码里已处理 Type-3 线格式、手续费迭代、签名

> 我们不提供远程签名接口。**节点的 `/util/transaction/sign` 要求把私钥通过 HTTP 传过去**，任何情况下都不要用。

4.4 广播与确认

```
POST /submit/transaction        # 二进制体，不是 hex
```

```
{"ret": 0, "hash": "..."}
```

确认流程：

1. `ret:0` → 已进内存池，**尚未确认**
2. 轮询 `GET /query/transaction?hash={哈希}`
3. 看 `confirm` 字段，`≥ 1` 表示已上链
4. 或直接核对 `GET /query/balance` 的余额变化

出块目标 300 秒。 充值 / 提现的风控确认数建议按这个节奏设计。当前链上实测约 5 分钟一块。

4.5 手续费与 gas

- **手续费**：按交易字节数计。用 `GET /query/fee/average` 拿 `purity`，`fee = purity × billing_size`。

提高 fee 会让字节数微增（fee 编码变长），所以要**迭代**：构建 → 读实际字节数 → 不足则重建。本仓库 `fee.js` 是这套迭代的实现，通常 1~3 轮收敛。

- **gas**：`gas_max` 字节，默认 `99` 即协议上限 `111,911`。交易所的转账类交易不碰合约，gas 消耗很低。

4.6 三个必知的硬限制

① VM gas 分三个独立桶，与 `gas_max` 无关

桶	上限
compute	18,009
resource	6,100
storage	111,911

超限时**提高 `gas_max` 完全无效。**错误形态：

```
OutOfGas(31): resource gas limit exceeded: used 6101 > limit 6100
```

② 单笔向合约存入最多 1 颗 HACD

每颗钻石的证书验签要在链上重算评分，约 3,000 resource，2 颗即 6,101 > 6,100。

交易所若提供「钻石换 hacds」服务，必须**拆成每笔一颗。**原子交易合并也不能绕过（共享同一 gas 预算）。

③ 沙箱调用每 IP 并发上限 = 2

读合约状态时不要并发，超出的请求会被直接拒绝（无排队）：

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

4.7 做市 / 报价需要的数据

需要的量	接口
池中 HAC	`sandboxcall` → `amm_reserves`（zhu，÷1e8 = HAC）
池中 hacds	`sandboxcall` → `amm_reserve_ds`（atoms）
现货价	`sandboxcall` → `pool_price` = `amm:hac × 1e8 / amm:ds`，即 **HAC/hacds**
总供给	`/query/balance?address={合约}&assets=true` 里 serial 2000 的 `supply`
可清算仓位	索引器 `GET /watchlist`

报价换算：

```
池价（HAC 每个 hacds）= pool_price / 1e8
例如 pool_price = 301355749 → 3.01355749 HAC/hacds
```

4.8 对账

链上恒等式，随时可核对：

> 用户持有 + 合约持有 = 总供给

三个数都能取到：

```
# 总供给
GET /query/balance?address={合约}&assets=true     → supply
# 合约持有
GET /query/balance?address={合约}&assets=true     → amount
# 用户持有
GET /query/balance?address={用户}&assets=true     → amount
```

实测样本（2026-10-21）：

项	atoms
总供给	`5963298700000000`
合约持有	`5963294700000000`
用户持有	`4000000000`
合约 + 用户	`5963298700000000` ✓

4.9 上币对接清单

- [ ] 资产身份按 **serial 2000** 匹配，同时校验 issuer
- [ ] `amount` 全程用大整数，绝不经过 double
- [ ] 展示给用户时除以 `1e8` 得到个数
- [ ] 充值识别按 `kind 17/19` 解析资产动作
- [ ] 提现构造 `TransferAssetFromTo(2000, amount, hotWallet, user)`
- [ ] 广播用二进制体 POST `/submit/transaction`
- [ ] 确认按 `confirm ≥ 1` 判，预留 5 分钟出块延迟
- [ ] 手续费迭代估算（`purity × size`）
- [ ] 沙箱读取串行化（每 IP 并发 ≤ 2）
- [ ] 若涉及钻石兑换，**一笔一颗**
- [ ] 对账用「用户 + 合约 = 总供给」恒等式

---

5. 错误处理

5.1 合约返回码

沙箱调用返回的 `ret_val` 非 0 时，是合约函数内部的 `return N`。常用映射：

函数	码	含义
通用	1	合约已暂停（`cfg:paused = 1`）
`swap_hac_to_ds`	11	HAC 积分不足
`add_liquidity`	2	某一侧为 0
`add_liquidity`	23	池非空但份额归零（防尺度重置）
`add_liquidity`	24	份额算出来为 0（比例太小）
`add_liquidity`	26	首注单侧超过上限 `1e18`
`remove_liquidity`	25	LP 份额不足
`loan_borrow`	40	超出可借额度（`(debt+hac)×2 > coll_hac`）
`loan_repay`	41	债务为 0
`loan_liquidate`	42	该地址无债务
`loan_liquidate`	43	未达清算线（不可清算）
`Set*`（管理员）	1	非管理员
`Set*`（管理员）	2	参数超范围
`PermitAsset`	11	待领取 hacds 不足

函数	码	含义
`PermitHACD`	12	兑换所需 hacds 不足且自动补足失败

完整表在 `wallet-app/contract-errors.js`。

5.2 节点 / 沙箱错误

按字符串匹配，常见几类：

原始信息	含义	处理
`invalid address argument` / `InvalidChecksum`	地址格式或校验位错	重新取完整地址
`balance ... is insufficient`	调用者余额不足付预扣 gas	沙箱调用传 `caller={合约地址}`
`sandbox_per_ip_concurrency_exceeded`	并发超限	串行化 + 重试
`resource gas limit exceeded`	resource 桶超限	拆笔；**提高 gas_max 无效**
`compute gas limit exceeded`	compute 桶超限	拆笔
`storage gas limit exceeded`	storage 桶超限	减少新键或降租期
`OutOfGas`	gas 不足	提高 gas_max
`CallNotExist`	合约没有这个函数	检查函数名
`NotFindContract`	找不到合约	检查合约地址
`state changed`	期间链上状态变了	重试
`transaction parse failed`	交易体解析失败	hex / 字节不完整
`ECONNREFUSED` / `timeout`	服务不可达	检查服务是否在跑

判别要点：这些错误大多是 **HTTP 200 +** `{"ret":1,"err":"..."}`，只看 HTTP 状态码会漏掉。

---

6. 快速参考

端点汇总

```
# 全节点 :8082
GET  /query/block/height/latest
GET  /query/fee/average
GET  /query/balance?address=&assets=true&asset_meta=true&diamonds=true
GET  /query/diamond?name=
GET  /query/contract/sandboxcall?contract=&function=&params=&caller=
GET  /query/block/intro?height=&tx_hash_list=true
GET  /query/transaction?hash=&action=true
GET  /query/contract/storage_fee
POST /util/transaction/check?description=true     # 二进制体
POST /submit/transaction                          # 二进制体

# 预言机 :8123
GET  /health
GET  /cert?name=

# 索引器 :8124
GET  /health
GET  /watchlist        # 客户端唯一需要的
GET  /snapshot
GET  /sync
GET  /watch/add?address=&note=
GET  /watch/remove?address=
```

常量

```
hacds serial         2000
hacds decimal        8
hacds 总供给          5963298700000000 atoms
1 hacds              100000000 atoms
合约                  VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
金库                  1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK
预言机账户            1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu
SCORE_SCALE          1e8（1 分 = 1 个 hacds）
COLL_RATIO           2（50% 抵押率，清算线 200）
AMM 手续费            0.5%（LP 50% / 质押 30% / 金库 20%）
借贷利率 / 清算奖励     年化 5% / 2%
证书长度              126 字节（消息 29 + 签名 97）
VM 三桶上限           compute 18,009 / resource 6,100 / storage 111,911
```

```
出块目标              300 秒
沙箱并发上限           每 IP 2
```

单位换算

```
atoms → hacds 个数 ：  amount / 1e8
hacds 个数 → atoms ：  count × 1e8
zhu   → HAC      ：  zhu / 1e8
池价（HAC/hacds）  ：  pool_price / 1e8
```